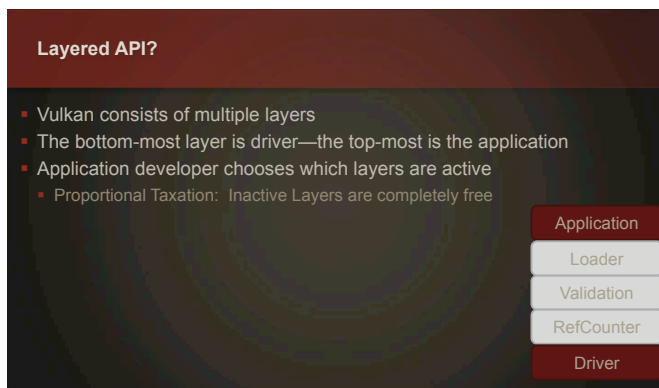


To help developers ship error-free code, Vulkan will ship with good tools to properly test the application during development. Vulkan, unlike OpenGL will ship with a full SDK.

Layered design

Some of the OpenGL features that have been removed in Vulkan are now available as additional layers over the core driver. For instance, there are layers available for validation, reference counting, debugging, checking for thread safety, trace capture and replay, and performance linting.



In Vulkan, disabled layers do not use up any CPU or GPU

Developers can create their own layers, and enable / disable any layer if needed. Third parties can create their own layers. For instance Steam's overlay that shows game information can be implemented as a Vulkan layer.

Bytecode Shaders

Shaders are one of the most important bits of GPU code while developing games but their current design leaves a lot to be desired. Currently there are two main languages to create shaders GLSL (for OpenGL) and HLSL (for DirectX).

Developers wanting to target both OpenGL and DirectX need to create the same shader code in both languages and ship them with the game. This leads to a duplication of effort in keeping two ver-

sions of the same code in sync. Or use automated code-conversion tools to output code in these languages.

Game developers also need to ship the source code of their shaders, which are then compiled by the graphics drivers at run time before being sent to the GPU. Not only does this cost in performance, it also causes issues for companies that want to keep their code private.

With Vulkan comes a new binary shader format called SPIR-V. This is a bytecode format that can be compiled to run on any GPU. Since it is a bytecode format, it removes the need for a full compiler in the driver, and reduces the performance impact of compiling shaders at run time. Also companies that want to keep their shaders private can now do so.

Of course developers won't directly write shaders in this binary format. Rather the idea is to support multiple source languages, which can all be compiled to the SPIR-V format.

Game developers can use the language of their choice, GLSL, HLSL, or some other language entirely – such as Apple's new Metal shader language. As long as there is a compiler for it that outputs SPIR-V.

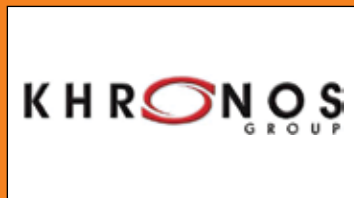
Conclusion

Thanks to AMD's donation of Mantle, Vulkan is well on its way to becoming a reality soon. EA's Frostbite 3 engine, already supports Mantle, and EA has committed to port it to Vulkan. Valve's Source 2 engine already supports Vulkan alpha, and their Steam Machines will also support Vulkan. Valve has also (in partnership with LunarG) developed a driver for Intel GPUs that supports Vulkan and will be kept in sync with the latest Vulkan specifications. This driver will be open sourced.

The Vulkan spec and drivers should be available this year itself, and the spec for SPIR-V is already available. A compiler for GLSL to SPIR-V is also available, and SPIR-V is supported by the newest OpenGL.

It will be interesting to see how major game companies, platform-holders and vendors get behind Vulkan and (if at all) current-generation consoles also take to Vulkan, but in the words of Valve's John McDonald, "Outlook good?". >>

*Multi-threads to look up



*Khronos Group

>>If you want to keep abreast of all developments in Vulkan, SPIR-V and other related technology, Khronos' website is where you should keep an eye on. Just forget everything and head over there.

<https://www.khronos.org>



*Mantle Programming Guide from AMD

>>While the Vulkan team works on a spec, the Mantle programming guide should serve as a good look at what's to come. We don't know, but can you foresee any surprises?

<http://dgit.in/MantleProg>



*Metal for Developers

>>Metal is the only complete low-level graphics API you can use today, albeit only for Apple devices. For developers trying to unlock the new iPhones' chip-level goodness, waste no time.

<http://dgit.in/MetalApple>